Week 5 — Paper notes: SARATHI-Serve (chunked prefills, stall-free batching)

Agrawal et al., *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve*, OSDI 2024.
Written from memory first, then checked against the paper. Corrections marked. Same method as Week 2.
*Term note: the paper says **TBT** (time-between-tokens) where I say ITL — same thing.*

Q1 — What is a generation stall? Who suffers?

Generation stalls mean high latency when a new request arrives while a decode phase is in progress. It happens in iteration-level (continuous) batching systems like Orca and vLLM, where the batch is re-decided every iteration and a new request can join at the next iteration boundary.

Who suffers: ❗ *not* the request being prefilled — its TTFT is fine. The ones **already decoding** suffer. When a big prefill lands in the same iteration as running decodes, the prefill saturates the GPU, the decodes wait, and their ITL/TBT spikes.

Correction to my first draft — I had the prioritization backwards.

- ❌ I wrote that these systems "prefer prefill prioritization, which increases throughput." That describes the **problem**, not the fix. Default vLLM/Orca **prioritize prefills** (admit a new prompt eagerly), and *that* is what *causes* the stall.
- ✅ SARATHI-Serve's contribution is to **stop** doing that: chunk the prefill and interleave the chunks with the decodes so no single iteration is prefill-dominated. It trades a little throughput for **stable ITL**, not the other way round.
- 🔵 Also: "new request joins the batch when an existing request is done" is not quite it. In iteration-level batching the batch is rebuilt **every iteration** — a new request joins at the next boundary, not only when an old one finishes.

Connects to my Week 3 data: my c8→c64 finding was "throughput flat, TTFT up 20×." That is this exact prefill/decode interference, measured on my own rig.

Q2 — Token budget per iteration, and prompts longer than it

The token budget per iteration is a fixed amount of tokens that can be processed in one iteration. Decode tokens are processed normally (~1 token each). When a new request comes in, its prompt is chunked and fitted into what's left of the budget; the overflow chunks wait for the next iterations.

If the prompt is longer than the remaining budget, it is split — this iteration's chunk goes now, the rest rides later iterations until the whole prompt is processed.

Correction to my worked example — the numbers didn't add up.

- ❌ I wrote "512 budget, 512 prompt, chunked into 4." A 512 prompt with a 512 budget **fits in one iteration** — no chunking needed. Chunking only happens when the prompt is longer than what's **left** of the budget.
- ✅ The case I was reaching for, made consistent — budget 512, 32 decodes already running:

  ```
  budget               = 512
  decodes this step    = 32 tokens (1 each)      -> 480 left for prefill
  new prompt           = 512 tokens              -> doesn't fit in 480
  chunk it:  480 now  +  32 next iteration
  this iteration total  = 32 decode + 480 prefill = 512  (exactly the budget)
  ```

  So decodes first, prefill fills the rest, overflow waits — my instinct was right, the arithmetic just has to sum to the budget.

Connects forward: this budget is `max_num_batched_tokens` in the vLLM scheduler (Week 6 code reading).

Q3 — What does stall-free cost?

Stall-free ensures decodes never experience a generation stall due to a co-running prefill chunk. It is **not free** — you pay in three places, plus one deeper cost I missed at first.

Cost 1 — higher TTFT. When the request doesn't fit the token budget, its whole prompt does not process in a single shot, so the first token arrives later. ✅ Confirmed: the paper's ablation shows chunked-prefills-only raises TTFT.

Cost 2 — KV-cache re-fetch. Each chunk re-reads all prior chunks' KV from GPU memory during attention. For N chunks, chunk i reads chunks $0 \ldots i-1$, so reads grow $\sim N(N-1)/2$. Computation stays the same; it's the repeated HBM reads that pile up. Stays modest — **~25% slower at chunk=512, near-zero at 2048** — because prefill stays **compute-bound** even at small chunks, so the extra reads hide behind compute. ✅ Both numbers confirmed (Yi-34B, Fig 14).

Cost 3 — tile quantization. GPUs do matmuls in fixed-size tiles; a chunk size that doesn't divide the tile wastes thread-block cycles, so the budget needs hardware profiling, not a round number. ✅ Concept confirmed. ⚠ **Verify:** I wrote "chunk 257 ~32% slower than 256." Mechanism right, exact 32% unconfirmed against the paper's figure — check or soften before quoting.

The deeper cost I first missed. Stall-free *forces* you to set a token budget, and that budget is a two-sided tension:

- too small → low arithmetic intensity + more KV re-reads → prefill overhead
- too large → the prefill chunk grows → decode stalls come back → higher ITL The paper's strongest claim: the optimal budget sits at the **linear-layer compute/memory crossover (the arithmetic-intensity knee)**, roughly **workload-independent**. So the cost is really: stall-free turns serving into a one-knob tuning problem, and the knob's right answer is set by the GPU, not the traffic.

Truck analogy. The shipment now takes many small trips instead of one big one. Two costs stack: the full order arrives later (**TTFT**), and every trip re-checks the manifest of everything already delivered before adding the new box (**KV re-read**). And the truck's size has to match the loading dock, not just any round number (**tile quantization / the budget**).

Does this predict my Week 3 anomaly? (my rule: hold the data against the paper)

20 requests queued with KV 94% empty. SARATHI explains half of it: queuing is driven by the **token budget**, not by KV capacity. A burst of prompts can't all clear the budget in one iteration, so they wait — while the KV pool is still nearly empty. The KV ceiling (97.5%) and the queue are unrelated, which is exactly what I measured. ✅ The paper predicts the *mechanism*; Week 6's scheduler code is where I confirm it line by line.

vllm Repo code reading to find so ans

Read these 2 files vllm/entrypoints/openai/api_server.py:
vllm/entrypoints/openai/chat_completion/serving.py:

I tried to find answers for these questions

Q. What does the server do to the messages list before the engine ever sees it?

Before seding to engine the message are sent to a funtion called create_chat_completion method that exist nthe api_router.py file which then utlizes the Chat(OpenAIServingChat) Instance which is in serving.py file

Q. Is the template applied to text, or to token ids?

The flow is is like this _create_chat_completion -> render_chat_request -> in here it says it returns convo that means it applies on the text not on the token ids.

Q. What is the streaming function's return type? What does that tell you about how FastAPI sends it?

It returns the type of AsyncGenerator, the router wraps that generator in StreamingResponse(content=generator, media_type="text/event-stream") — Starlette iterates the generator lazily and writes each yielded string to the response as it's produced, rather than waiting for the whole thing and sending it in one shot.

Week 5, Wednesday: text becomes an engine request

One sentence to remember: text is tokenised in the frontend process, and only token IDs cross to the engine, packed in an `EngineCoreRequest` and sent as msgpack bytes over ZMQ.

Line numbers are from commit 2cf0a69 and will drift. Function names are the stable anchors.

Q1. What object is sent to the engine, and which fields matter most?

Answer: `EngineCoreRequest`, a `msgspec.Struct`.

- Defined in `vllm/v1/engine/__init__.py` (class `EngineCoreRequest`, line 100).
- Built in `InputProcessor.process_inputs` (`input_processor.py`, line 384).
- 21 fields. 15 are set when it is built; 6 have defaults and are filled later.

The five that matter most, and why:

Field	Why the engine needs it
`request_id`	Routes outputs back to the right caller
`prompt_token_ids`	The actual input. There is no text field at all
`sampling_params`	How to generate and when to stop (`max_tokens`, temperature)
`arrival_time`	Start of the TTFT and end-to-end metrics
`priority`	Queue order under priority scheduling, with `arrival_time` as tie-break

Fields filled in later (same object, mutated along the way):

- `external_req_id`: `assign_request_id`, called from `AsyncLLM.add_request`
- `reasoning_ended`, `reasoning_parser_kwargs`: `AsyncLLM.add_request`
- `client_index`: `AsyncMPClient.add_request_async`, just before sending
- `current_wave`: data-parallel client only
- `abort_immediately`: a special reject path in `async_llm.py` that builds its own request

My mistake: I treated the definition and the construction as two different objects, and I ranked `cache_salt` and `session_id` above the payload.

Note: tokenisation is no longer in `input_processor.py` on the normal path. It happens earlier in the Renderer (`tokenize_prompts` in `vllm/renderers/base.py`). `InputProcessor` now mostly validates and packages.

Q2. What transport carries the request to the engine process?

Answer: ZMQ (ZeroMQ) moves the bytes. msgspec (msgpack format) turns the object into bytes and back. These are two separate jobs.

The path:

1. `AsyncLLM._add_request` calls `engine_core.add_request_async(request)`
2. `AsyncMPClient._send_input` encodes it with `MsgpackEncoder`
3. `input_socket.send_multipart(...)` sends `(engine_id, request_type, *frames)`, where the type for a new request is `ADD = b"\x00"`
4. Process boundary
5. `EngineCoreProc.process_input_sockets` (an IO thread) decodes it with `MsgpackDecoder(EngineCoreRequest)`
6. `preprocess_add_request` converts it to a `Request`
7. It goes onto `input_queue` for the busy loop

Sockets: requests go frontend `ROUTER` to engine `DEALER`. Outputs come back on a separate pair, engine `PUSH` to frontend `PULL`.

Why a separate process: the GIL. The frontend does HTTP, tokenising, detokenising and JSON. The engine runs the scheduling and model loop. In one process, each would block the other.

Exception: `InprocClient` calls the engine directly, with no process and no ZMQ. `EngineCoreClient.make_client` chooses which client to use.

My mistake: I said msgspec "validates" the request. Its main job is serialising. Typed decoding does reject bad data, but that is a side effect.

Q3. Where is the server-side arrival time recorded?

Answer: in the Renderer, not in `add_request`.

- Recorded: `arrival_time = time.time()` at the top of `render_cmpl` and `render_chat` (`vllm/renderers/base.py`, lines 993 and 1044), before templating and tokenisation.
- Carried: stored as `engine_input["arrival_time"]` in `process_for_engine`.
- Read: `process_inputs` picks it up with `prompt.get("arrival_time", ...)` and copies it into `EngineCoreRequest.arrival_time`.

Traps:

- `AsyncLLM.add_request` only forwards a parameter. On the normal path it is `None`, because `generate()` never passes `arrival_time`.
- The two `time.time()` calls inside `process_inputs` (lines 292 and 303) are fallbacks. Line 303 is in the deprecated raw-prompt branch.

TTFT link:

- vLLM's TTFT metric = first token processed in the frontend minus `arrival_time` (`vllm/v1/metrics/stats.py`, line 393).
- My client-side TTFT = (gap from client `st` to server `arrival_time`) + vLLM's TTFT + the response travelling back.
- That first gap is network plus HTTP parsing and validation. vLLM's own metric cannot see it.

My mistake: I searched for the name `arrival_time` and stopped at the first place it appeared. I should have searched for where the value is created.

Diagram

```
[1 API server + Renderer]
      |  EngineInput (dict: prompt_token_ids, arrival_time, ...)
      v
[2 AsyncLLM + InputProcessor]
      |  EngineCoreRequest (msgpack bytes over ZMQ)
      v
[3 EngineCore process]  ->  Request (for the scheduler)
```

Self-test (cover the answers above)

1. Does the engine process ever see the prompt text? Why not?
2. Which library moves the bytes, and which one makes the bytes?
3. `add_request` has an `arrival_time` parameter. What is its value on a normal chat request, and where does the real value come from?
4. Name two fields that are not set in `process_inputs`, and where they are set.
5. What is included in my client TTFT that vLLM's TTFT metric leaves out?

Reading habits to keep

- Trace the value, not the name: find where it is written, not where it is mentioned.
- For any object, find four places: defined, constructed, modified, sent.
- Rank a field by who reads it on the other side.
- A deprecation warning marks the old path. Follow the other one.

Thursday read code repo

1. step() does three things in order

Answer (core.py:583-613):

Schedule: self.scheduler.schedule(...) decides which requests run this step, how many tokens each gets, and which KV blocks they use. It returns a SchedulerOutput. Execute: self.model_executor.execute_model(scheduler_output, non_block=True) runs the forward pass, and sample_tokens(grammar_output) picks the next token ids. Update: self.scheduler.update_from_output(scheduler_output, model_output) appends the new tokens to each request, checks stop conditions, and builds the EngineCoreOutputs that go back to the frontend.

2. Request states and transitions

The states (request.py:351-367): WAITING, WAITING_FOR_STRUCTURED_OUTPUT_GRAMMAR, WAITING_FOR_REMOTE_KVS, WAITING_FOR_STREAMING_REQ, RUNNING, PREEMPTED, then six FINISHED_* states.

The order matters: is_finished() is status > PREEMPTED, which is why the comment says anything after PREEMPTED counts as finished.

new request │ (structured output?) ├──────────► WAITING_FOR_STRUCTURED_OUTPUT_GRAMMAR │ │ grammar compiled │ compile error ▼ ▼ ▼ WAITING ◄──────────┘ FINISHED_ERROR │ ▲ │ └──────── WAITING_FOR_REMOTE_KVS ◄── (from WAITING or PREEMPTED, │ │ async KV load started) │ └──► back to WAITING, or PREEMPTED if it was preempted before ▼ RUNNING ◄──────────── PREEMPTED │ └── out of KV blocks ──► PREEMPTED │ ├─ EOS / stop token ──────► FINISHED_STOPPED ├─ max_tokens / max len ──► FINISHED_LENGTH_CAPPED ├─ repetition detected ──► FINISHED_REPETITION ├─ grammar rejects token ──► FINISHED_ERROR └─ (resumable session) ──► WAITING_FOR_STREAMING_REQ ──► WAITING (next chunk) └─► FINISHED_ABORTED (session ends)

any unfinished state ──► FINISHED_ABORTED (client abort, shutdown)

3. What num_computed_tokens counts, and when it goes down

Answer: it is how many tokens at the start of the request's sequence (prompt plus output) already have their KV cache entries, so they need no forward pass. Each step the scheduler asks for the rest: num_new_tokens = num_tokens - num_computed_tokens (scheduler.py:932).

It goes up in two places:

At admission: it is set to the prefix-cache hit, local plus external (scheduler.py:881-883, scheduler.py:1136). After scheduling: += num_scheduled_token, before the GPU has finished (scheduler.py:1379-1393). The comment there gives the reason: a long prefill can be scheduled again in the very next step. So it is optimistic: it means "scheduled to be computed", not "confirmed computed".

It goes down in four cases:

Preemption (the answer the hint wants): the request's blocks are freed, so the counter is reset to 0 (scheduler.py:1352-1356). On resume, the prefix-cache lookup runs again because the counter is 0 (scheduler.py:811), so some blocks may be recovered without recomputing. Rejected speculative tokens: -= num_rejected, because the count was advanced for draft tokens that turned out wrong (scheduler.py:1844-1846). Invalid or failed KV blocks: truncated to the first bad block, idx * block_size (scheduler.py:2908). Full remote hit on the whole prompt: set to num_tokens - 1, so the last token is recomputed to produce logits to sample from (scheduler.py:2761-2762).

4. Why detokenisation is incremental, and why a token can produce no text

Where it happens: not in the engine core. The core sends only token ids. The frontend's OutputProcessor.process_outputs calls detokenizer.update(...) (output_processor.py:669), which calls decode_next once per token (detokenizer.py:118-120).

Why incremental:

Streaming needs the new text after every token. Decoding the whole output each step would cost more and more as the output grows. Decoding each token alone and joining the strings is wrong, because the text of a token depends on its neighbours (leading spaces, and characters split across tokens). So the detokenizer keeps a small window. In the slow path (detokenizer_utils.py:241-268) it decodes the window without the new token (prefix_text) and with it (new_text), and emits only the difference. The fast path does the same job with the tokenizers library's DecodeStream.

Why one token id can give no text:

Incomplete UTF-8 character (the main one). In byte-level BPE a token is a run of bytes, not characters. Urdu "ا" is two bytes (D8 A7) and 😀 is four (F0 9F 98 80). If the model emits only the first part, decoding gives "�". The code detects this and returns "" without moving its offsets (detokenizer_utils.py:260-265), so the next token is decoded together with the held one. The fast path returns None, turned into "" at detokenizer.py:222. Special tokens such as EOS, when skip_special_tokens is on. Stop strings. The last max(len(stop)) - 1 characters are held back in case they are the start of a stop string (detokenizer.py:85-90, detokenizer.py:149-165). The text exists but is not released yet. Out-of-vocabulary id: decoded as "" (detokenizer_utils.py:218-231).